Lab 12

MQTT and RabbitMQ: Brokers, QoS and Topics

Communication Technologies for the Internet of Things | Lecture 12

Aim and context

Lecture 12 introduces the message-oriented paradigm, with MQTT and AMQP. This lab installs two brokers, Mosquitto (MQTT) and RabbitMQ (AMQP), on a virtual machine, and makes decoupling, QoS levels, retained messages, topics and the exchanges of a queue and of a topic exchange observable.

The work does not require access to the course server. It is meant to be done during the class, at each student's pace, and is **not an assignment to hand in**.

> **Note**
>
> The results shown are examples: identifiers, times and version numbers differ from computer to computer.

Learning outcomes

- Publish and subscribe MQTT messages, with hierarchical topics and wildcards.
- Distinguish the QoS 0, 1 and 2 levels by the exchanges observed.
- Demonstrate a retained message and a last will.
- Write a producer and a consumer with Python libraries.
- Distinguish the queue model from the topic exchange model in RabbitMQ.

Requirements

- A computer running Windows, macOS or Linux, with about 2 GB of free memory.
- An Internet connection, needed to fetch the image and packages.
- Estimated duration: 75 to 100 minutes.

How to use this guide

Each step presents, in this order, the aim, the context, the theoretical background, the commands to run and the expected result. Read the whole step before running the first command. The boxes always have the same meaning:

Command to run

```
Command to run in the terminal.
```

Expected result (values may vary)

```
Result that should be displayed.
```

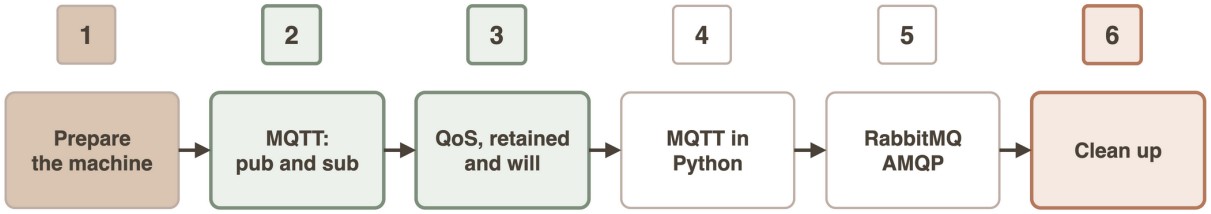
Analyse

A moment to pause and interpret the result, on your own or in discussion with colleagues.

Possible error and fix

An error that may occur and how to fix it.

Overview

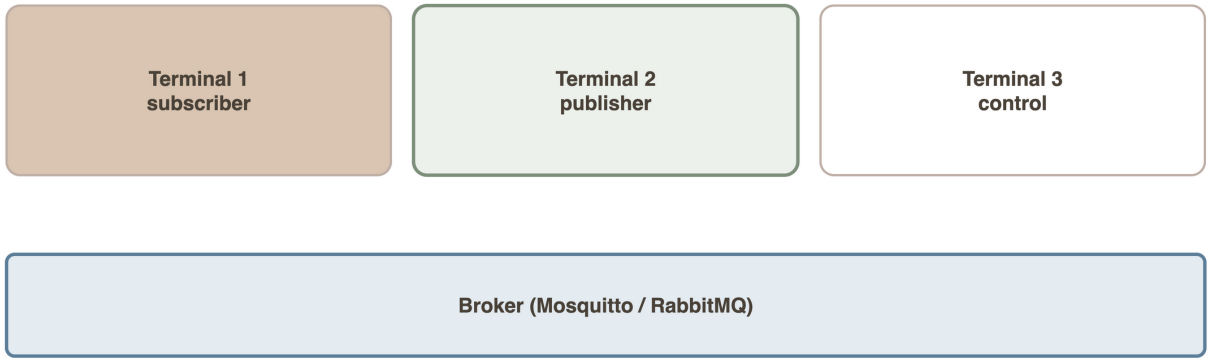


Two brokers on the same machine: Mosquitto (MQTT) and RabbitMQ (AMQP)

How to read the figure

The six steps read from left to right. The first prepares the machine. Steps 2 and 3, in green, explore MQTT with the command-line tools. Step 4 repeats the idea in Python and step 5 moves to AMQP with RabbitMQ. Step 6 closes the work.

Several steps need three terminals, all connected to the same virtual machine with `multipass shell tciot-mom`.



All three terminals connect to the same VM: multipass shell tciot-mom

How to read the figure

Terminal 1, in beige, stays subscribed and should remain open. Terminal 2, in green, publishes. Terminal 3, in white, is for control commands, such as killing processes. At the bottom is the broker, common to all.

Step 1 — Preparing the machine

Aim. Create an Ubuntu virtual machine and install the two brokers and the Python libraries.

Theoretical background

Mosquitto is a lightweight MQTT broker, and RabbitMQ is an AMQP broker written in Erlang. Both run as system services and listen on well-known ports: 1883 for MQTT and 5672 for AMQP. By default, both accept only local connections, which suits this lab.

1.1 Creating the machine and installing

Command to run

```
multipass launch 22.04 --name tciot-mom --cpus 2 --memory 2G --disk 8G
multipass shell tciot-mom
```

Command to run

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients rabbitmq-server python3-venv
python3 -m venv ~/mq
~/mq/bin/pip install paho-mqtt pika
```

Command to run

```
systemctl is-active mosquitto rabbitmq-server
```

Expected result (values may vary)

```
active
active
```

Possible error and fix

launch failed: not enough free memory. Close other applications and try again. If the machine already exists, `multipass delete tciot-mom`, `multipass purge` and repeat. **inactive** for RabbitMQ: startup takes about half a minute. Wait and repeat the command.

1.2 The Python programs

Create, in the home folder, the following two files. They are also in the `code` folder of the lecture material.

Contents of file `mqtt_demo.py`

```
"""MQTT demo (Lecture 12): publisher and subscriber with paho-mqtt.

Run in two terminals:
  python mqtt_demo.py sub          # subscribes to casa/+/temperatura
  python mqtt_demo.py pub [qos]    # publishes 5 readings
"""
import json
import random
import sys
import time

import paho.mqtt.client as mqtt

BROKER = "localhost"
TOPIC = "casa/sala/temperatura"
FILTER = "casa/+/temperatura"


def client():
    return mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)


def sub():
    def on_message(c, userdata, msg):
        print(f"{msg.topic} qos={msg.qos} retained={msg.retain} {msg.payload.
    decode()}")

    c = client()
    c.on_message = on_message
    c.connect(BROKER)
    c.subscribe(FILTER, qos=1)
    c.loop_forever()


def pub(qos=1):
    c = client()
    c.connect(BROKER)
    c.loop_start()
    for n in range(5):
        value = round(20 + random.random() * 3, 1)
        info = c.publish(TOPIC, json.dumps({"n": n, "value": value}), qos=qos)
```

```
        info.wait_for_publish()
        print(f"published n={n} value={value} qos={qos}")
        time.sleep(1)
    c.loop_stop()
    c.disconnect()


if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "sub":
        sub()
    else:
        pub(int(sys.argv[2]) if len(sys.argv) > 2 else 1)
```

Contents of file `amqp_demo.py`

```
"""AMQP demo (Lecture 12): work queue and topic exchange with pika and RabbitMQ.

Terminals:
  python amqp_demo.py worker NAME        # competing consumer on the queue '
    tarefas'
  python amqp_demo.py send N             # sends N tasks to the queue
  python amqp_demo.py listen PATTERN     # binds a private queue to the exchange '
    casa' (topic)
  python amqp_demo.py emit KEY TEXT      # publishes TEXT with routing key KEY on
    'casa'
"""
import sys

import pika

HOST = "localhost"


def channel():
    conn = pika.BlockingConnection(pika.ConnectionParameters(HOST))
    return conn, conn.channel()


def worker(name):
    conn, ch = channel()
    ch.queue_declare(queue="tarefas", durable=True)
    ch.basic_qos(prefetch_count=1)

    def on_msg(c, method, props, body):
        print(f"[{name}] processing {body.decode()}")
        c.basic_ack(delivery_tag=method.delivery_tag)
```

```
    ch.basic_consume(queue="tarefas", on_message_callback=on_msg)
    ch.start_consuming()


def send(n):
    conn, ch = channel()
    ch.queue_declare(queue="tarefas", durable=True)
    for i in range(1, n + 1):
        ch.basic_publish("", "tarefas", f"task {i}".encode(),
                         pika.BasicProperties(delivery_mode=pika.DeliveryMode.
    Persistent))
        print(f"sent task {i}")
    conn.close()


def listen(pattern):
    conn, ch = channel()
    ch.exchange_declare(exchange="casa", exchange_type="topic")
    q = ch.queue_declare(queue="", exclusive=True).method.queue
    ch.queue_bind(exchange="casa", queue=q, routing_key=pattern)

    def on_msg(c, method, props, body):
        print(f"{method.routing_key}: {body.decode()}")

    ch.basic_consume(queue=q, on_message_callback=on_msg, auto_ack=True)
    ch.start_consuming()


def emit(key, text):
    conn, ch = channel()
    ch.exchange_declare(exchange="casa", exchange_type="topic")
    ch.basic_publish("casa", key, text.encode())
    print(f"published {key}: {text}")
    conn.close()


if __name__ == "__main__":
    cmd = sys.argv[1]
    {"worker": lambda: worker(sys.argv[2]), "send": lambda: send(int(sys.argv[2]))
    ,
     "listen": lambda: listen(sys.argv[2]), "emit": lambda: emit(sys.argv[2], sys.
    argv[3])}[cmd]()
```

Step 2 — MQTT with a publisher and a subscriber

Aim. Publish and subscribe messages by topic and observe decoupling.

Theoretical background

In an MQTT broker, topics are hierarchical strings separated by slashes. A subscriber gives a filter, which may contain the `+` sign, which replaces one level, and the hash `#`, which replaces all levels below. The publisher does not know the subscribers: it publishes on the topic and the broker delivers to whoever has a matching filter.

To analyse in this step: the publisher and the subscriber exchange no addresses and are not connected to each other.

In terminal 1, subscribe to everything below `casa`:

Command to run

```
mosquitto_sub -h localhost -t 'casa/#' -v
```

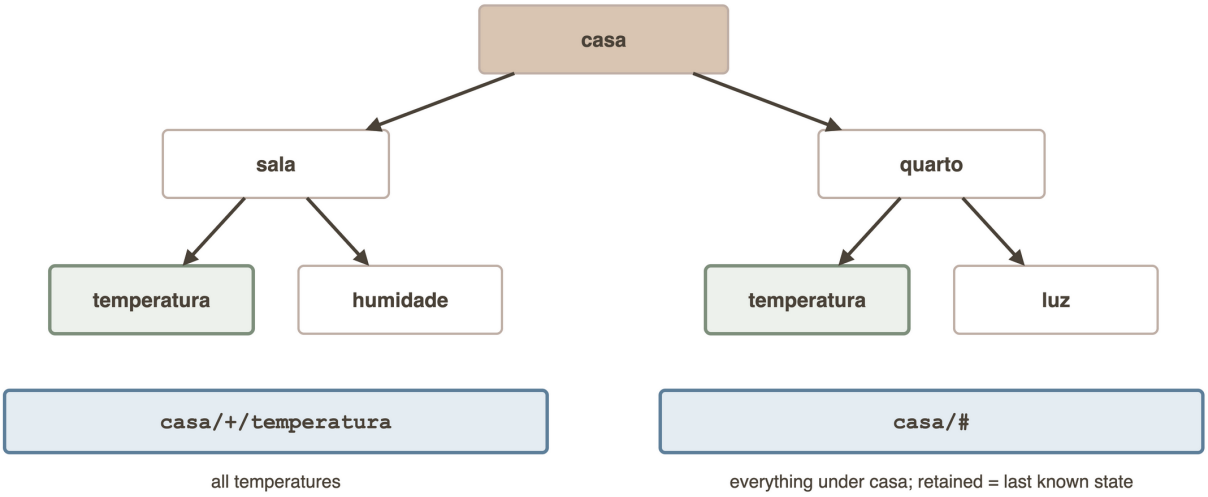
In terminal 2, publish two messages:

Command to run

```
mosquitto_pub -h localhost -t casa/sala/temperatura -m 21.5
mosquitto_pub -h localhost -t casa/quarto/luz -m on
```

Expected result (values may vary)

```
casa/sala/temperatura 21.5
casa/quarto/luz on
```



How to read the figure

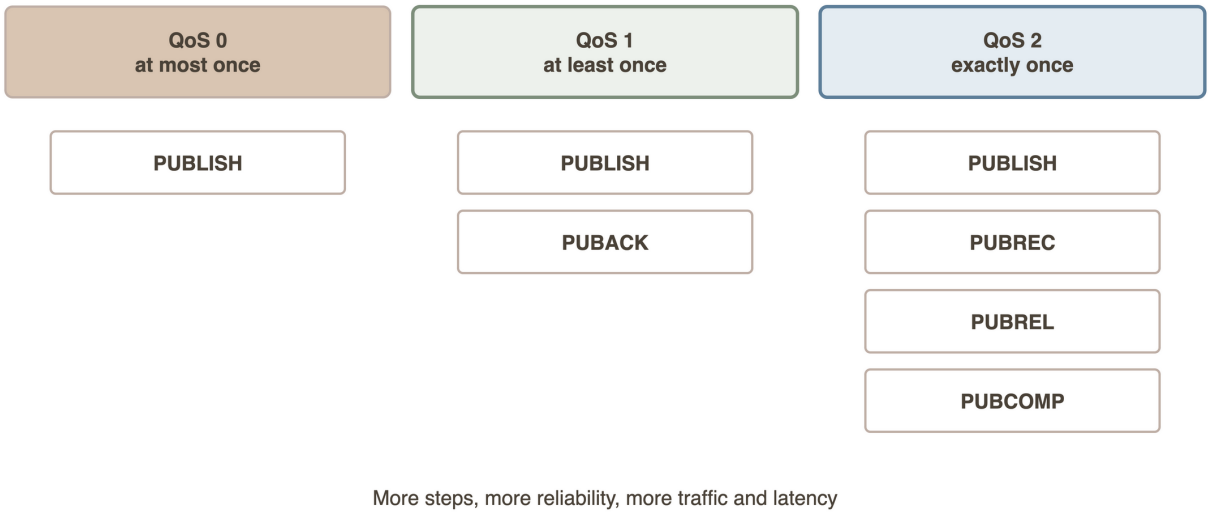
The figure shows the topic tree. The subscription `casa/#` catches everything, and `casa/+/temperatura` catches the temperatures of any room. The two messages published above are caught by the first.

Analyse

End the subscriber and subscribe again with `casa/+/temperatura`. Which messages do you receive now? What happened to the light message?

Step 3 — QoS, retained messages and last will

Aim. Observe the exchanges of each QoS level, the effect of a retained message and a client's last will.



How to read the figure

Each column is a QoS level and each box a message exchanged. Level 0 has a single message, level 1 adds an acknowledgement and level 2 uses four messages.

Theoretical background

QoS is requested by whoever publishes and by whoever subscribes, and the effective level is the lower of the two. A retained message is kept by the broker and delivered to whoever subscribes later. The last will is a message the client gives the broker when it connects, and which the broker publishes if the connection ends without a normal DISCONNECT.

3.1 Exchanges at each QoS

Publish with `-d`, which shows the protocol packets:

Command to run

```
mosquitto_pub -d -q 0 -t casa/sala/temperatura -m 21.5
mosquitto_pub -d -q 1 -t casa/sala/temperatura -m 21.5
mosquitto_pub -d -q 2 -t casa/sala/temperatura -m 21.5
```

Expected result (values may vary)

```
(q0) sending PUBLISH (d0, q0, r0, m1, 'casa/sala/temperatura', ... (4 bytes))
(q1) sending PUBLISH (d0, q1, r0, m1, ...)  received PUBACK (Mid: 1, RC:0)
(q2) sending PUBLISH (d0, q2, r0, m1, ...)  received PUBREC (Mid: 1, RC:0)
     sending PUBREL (Mid: 1)  received PUBCOMP (Mid: 1, RC:0)
```

3.2 Retained message

Command to run

```
mosquitto_pub -r -t casa/sala/temperatura -m 21.5
mosquitto_sub -t casa/sala/temperatura -C 1 -v
```

Expected result (values may vary)

```
casa/sala/temperatura 21.5
```

The second command ends immediately, because the broker delivers the retained message as soon as the subscription is made. To clear it: `mosquitto_pub -r -n -t casa/sala/temperatura`.

3.3 Last will

In terminal 1, subscribe to the state:

Command to run

```
mosquitto_sub -t estado/sensor1 -v
```

In terminal 2, connect a client with a will:

Command to run

```
mosquitto_sub -t qualquer --will-topic estado/sensor1 --will-payload offline
```

In terminal 3, kill the client abruptly:

Command to run

```
pkill -9 -f 'will-topic'
```

Expected result (values may vary)

```
estado/sensor1 offline
```

Analyse

Who published the message `offline`? Why does the publication only happen when the connection ends abnormally? How would this be used, combined with retained messages, to represent the presence of a sensor?

Step 4 — MQTT in Python

Aim. Write a producer and a consumer with the `paho-mqtt` library.

Theoretical background

An MQTT application connects to the broker, subscribes to filters and handles each message in a callback function. The producer publishes with a QoS of choice and, at QoS 1, waits for acknowledgement with `wait_for_publish`. The example consumer subscribes to `casa/+/temperatura` with QoS 1 and prints the topic, the QoS and the retained flag.

In terminal 1:

Command to run

```
~/mq/bin/python mqtt_demo.py sub
```

In terminal 2:

Command to run

```
~/mq/bin/python mqtt_demo.py pub 1
```

Expected result (values may vary)

```
published n=0 value=22.3 qos=1
published n=1 value=21.1 qos=1
...
```

Expected result (values may vary)

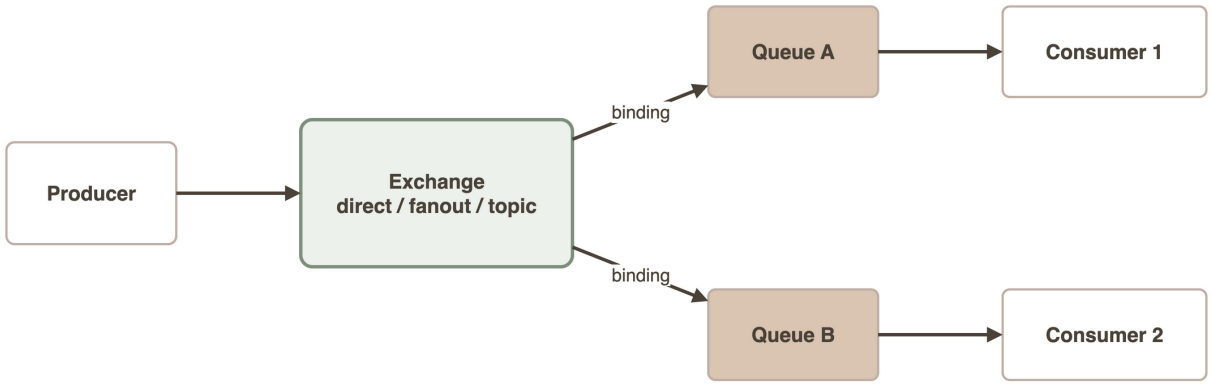
```
casa/sala/temperatura qos=1 retained=False {"n": 0, "value": 22.3}
casa/sala/temperatura qos=1 retained=False {"n": 1, "value": 21.1}
```

Analyse

Change the publisher's QoS to 0 and repeat. What difference do you notice at the consumer? How many consumers can be added without changing the producer?

Step 5 — AMQP with RabbitMQ

Aim. Observe a queue with competing consumers and a topic exchange.



The exchange decides, by type and bindings, which queues the message goes to

How to read the figure

The producer's message reaches the exchange, which routes it to the queues according to the bindings. On each queue, messages are delivered to consumers in order. On the task queue, each message goes to a single consumer.

Theoretical background

On a shared queue, RabbitMQ distributes messages to consumers in order, and with `prefetch_count=1` gives a new message only to whoever finished the previous one. On a *topic* exchange, the routing key is a sequence of words separated by dots, and bindings use `*`, which replaces one word, and `#`, which replaces zero or more. Note that the separator is the dot, unlike MQTT.

5.1 Task queue

In terminals 1 and 2, start two workers:

Command to run

```
~/mq/bin/python amqp_demo.py worker A
~/mq/bin/python amqp_demo.py worker B
```

In terminal 3, send six tasks:

Command to run

```
~/mq/bin/python amqp_demo.py send 6
```

Expected result (values may vary)

```
[A] processing task 1
[B] processing task 2
[A] processing task 3
[B] processing task 4
[A] processing task 5
[B] processing task 6
```

5.2 Topic exchange

Stop the workers. In terminals 1 and 2, start two listeners with different patterns:

Command to run

```
~/mq/bin/python amqp_demo.py listen 'casa.*.temperatura'
~/mq/bin/python amqp_demo.py listen 'casa.#'
```

In terminal 3:

Command to run

```
~/mq/bin/python amqp_demo.py emit casa.sala.temperatura 21.5
~/mq/bin/python amqp_demo.py emit casa.quarto.luz on
```

Expected result (values may vary)

```
casa.sala.temperatura: 21.5          (listener 1 and listener 2)
casa.quarto.luz: on                    (listener 2 only)
```

5.3 Management interface

Command to run

```
sudo rabbitmq-plugins enable rabbitmq_management
curl -s -u guest:guest http://localhost:15672/api/overview | head -c 200
sudo rabbitmqctl list_queues name messages consumers
```

Analyse

How many consumers does the task queue have after the workers are stopped? What would happen to messages sent while no worker was connected, and why?

Possible error and fix

Connection refused on RabbitMQ: the service is still starting. `sudo systemctl status rabbitmq-server` and wait. The `guest` user can only connect from the machine itself.

Step 6 — Cleaning up

Command to run

```
exit
multipass delete tciot-mom
multipass purge
```

Review questions

1. Why do the MQTT publisher and subscriber not need to know each other?
2. What is the difference between QoS 1 and QoS 2 in the messages exchanged?
3. What use is the last will on a battery-powered sensor?
4. What is the difference between two consumers on the same queue and two listeners bound to a topic exchange?
5. What separator do MQTT topics use and what separator do AMQP routing keys use?

Summary

- The **broker** decouples producers and consumers in space, in time and in synchronisation.
- MQTT **QoS** is chosen per message and changes the number of exchanges.
- **Retained messages** and the **last will** give state and presence to whoever arrives later.
- In **AMQP**, the exchange and the bindings decide where the message goes, and queues share the work.